Import Needed Libraries

```
import os
import cv2
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import tensorflow as tf
import keras
from sklearn.metrics import confusion_matrix , classification_report
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers import Adam , Adamax
from sklearn.utils import shuffle
from sklearn.model_selection import train_test_split
from tqdm import tqdm
import warnings
warnings.filterwarnings("ignore")
```

Reading Dataset And Get Info

Define Constraints

Kaggle API Setup (Reminder)

Before proceeding, please ensure you have set your `KAGGLE_USERNAME` and `KAGGLE_KEY` in Colab secrets as described previously. If you haven't, please follow the steps:

1. **Generate Kaggle API Token**: Go to your Kaggle account page (Profile -> Account), scroll down to the 'API' section, and click 'Create New API Token'. This will download a `kaggle.json` file.
2. **Upload `kaggle.json` to Colab Secrets**:
    - In Colab, click on the '🔑' (Secrets) icon on the left sidebar.
    - Click 'Add new secret'.
    - For the 'Name' field, type `KAGGLE_USERNAME` and for 'Value', open your `kaggle.json` file and copy the value associated with the key `username`. Then click 'Save secret'.
    - Repeat this process for `KAGGLE_KEY`, copying the value associated with `key` from `kaggle.json`.
3. **Enable access to `KAGGLE_USERNAME` and `KAGGLE_KEY`** for this notebook by toggling the switch next to them in the Secrets panel.

```
# Install the Kaggle library
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.4.0)
Requirement already satisfied: jupytext in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.19.4)
Requirement already satisfied: kagglesdk<1.0,>=0.1.20 in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.1.23)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaggle) (26.2)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: markdown-it-py>=1.0 in /usr/local/lib/python3.12/dist-packages (from jupytext->kaggle) (4.2.0
Requirement already satisfied: mdit-py-plugins in /usr/local/lib/python3.12/dist-packages (from jupytext->kaggle) (0.6.1)
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/dist-packages (from jupytext->kaggle) (5.10.4)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from jupytext->kaggle) (6.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->kaggle) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages (from python-slugify->kaggle)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.18)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (2026.6
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=1.0->jupytext->ka
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupytext->kag
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupytext->kaggle)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyte
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupytext->kaggle) (
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jup
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonsch
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbforma
```

```
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->j
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core!=5.0.*,>=4.12
Requirement already satisfied: typing-extensions>=4.4.0 in /usr/local/lib/python3.12/dist-packages (from referencing>=0.28.4
```

```
# Import necessary libraries
import os
from google.colab import userdata

# Set Kaggle API credentials from Colab secrets
os.environ['KAGGLE_USERNAME'] = userdata.get('KAGGLE_USERNAME')
os.environ['KAGGLE_KEY'] = userdata.get('KAGGLE_KEY')

# Verify if the credentials are set (optional, for debugging)
print("KAGGLE_USERNAME set.")
print("KAGGL_KEY set.")
```

```
KAGGLE_USERNAME set.
KAGGL_KEY set.
```

Now, let's download the correct dataset: `gunavenkatdoddi/eye-diseases-classification`.

```
# Create a directory for Kaggle datasets if it doesn't exist
!mkdir -p /content/kaggle_data

# Download the dataset from Kaggle
# The dataset name format is 'kaggle_username/dataset_name'
!kaggle datasets download -d gunavenkatdoddi/eye-diseases-classification -p /content/kaggle_data
```

```
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest ve
Dataset URL: https://www.kaggle.com/datasets/gunavenkatdoddi/eye-diseases-classification
License(s): ODbL-1.0
Downloading eye-diseases-classification.zip to /content/kaggle_data
100% 736M/736M [00:36<00:00, 21.1MB/s]
```

```
# Unzip the dataset. It will extract into '/content/kaggle_data/dataset'
!unzip -q /content/kaggle_data/eye-diseases-classification.zip -d /content/kaggle_data

# List the contents to verify
print("Contents of /content/kaggle_data:")
!ls -R /content/kaggle_data
```

```
2533_left.jpg   2752_right.jpg  2988_left.jpg   3221_right.jpg  3485_left.jpg
2533_right.jpg  2755_left.jpg   2988_right.jpg  3222_left.jpg   4290_right.jpg
2535_left.jpg   2755_right.jpg  2991_left.jpg   3222_right.jpg  4637_left.jpg
2535_right.jpg  2756_right.jpg  2991_right.jpg  3223_left.jpg   4637_right.jpg
2538_left.jpg   2761_left.jpg   2992_left.jpg   3223_right.jpg  530_left.jpg
2538_right.jpg  2761_right.jpg  2992_right.jpg  3226_left.jpg   530_right.jpg
2540_left.jpg   2763_left.jpg   2996_left.jpg   3226_right.jpg  695_left.jpg
2540_right.jpg  2763_right.jpg  2996_right.jpg  3227_left.jpg   695_right.jpg
2542_left.jpg   2764_left.jpg   3000_left.jpg   3227_right.jpg  84_left.jpg
2542_right.jpg  2764_right.jpg  3000_right.jpg  3232_left.jpg   84_right.jpg
2543_left.jpg   2767_left.jpg   3001_left.jpg   3232_right.jpg  8_left.jpg
2543_right.jpg  2767_right.jpg  3001_right.jpg  3234_left.jpg   8_right.jpg
2547_left.jpg   2768_left.jpg   3004_left.jpg   3234_right.jpg  939_left.jpg
2547_right.jpg  2768_right.jpg  3004_right.jpg  3237_left.jpg   939_right.jpg
2548_left.jpg   2770_left.jpg   3007_left.jpg   3237_right.jpg  951_left.jpg
2548_right.jpg  2770_right.jpg  3007_right.jpg  3238_right.jpg  951_right.jpg
2554_left.jpg   2771_left.jpg   3008_left.jpg   3240_right.jpg
```

```
IMAGE_SIZE = (224,224)
BATCH_SIZE = 32
CHANNELS = 3
```

Load The Data

```
DATASET_PATH = '/content/kaggle_data/dataset'
Class_Names = os.listdir(DATASET_PATH)
Class_Names_Dict = {name:idx for idx,name in enumerate(Class_Names)}
print(Class_Names_Dict)
```

```
{'diabetic_retinopathy': 0, 'glaucoma': 1, 'cataract': 2, 'normal': 3}
```

```
def Load_Data():
    Images = []
    Labels = []

    for folder in os.listdir(DATASET_PATH):
        print(f"Load Images from {folder} File")
        for file in tqdm(os.listdir(os.path.join(DATASET_PATH,folder))):
            image_path = os.path.join(os.path.join(DATASET_PATH,folder),file)
            image = cv2.imread(image_path)
            image = cv2.resize(image , IMAGE_SIZE)
            image = cv2.cvtColor(image , cv2.COLOR_BGR2RGB)

            Images.append(image)
            Labels.append(Class_Names_Dict[folder])


    Images = np.array(Images , dtype='float32')
    Labels = np.array(Labels , dtype='int32')

    return (Images , Labels)

Images , Labels = Load_Data()
```

```
Load Images from diabetic_retinopathy File
100%|██████████| 1098/1098 [00:02<00:00, 490.42it/s]
Load Images from glaucoma File
100%|██████████| 1007/1007 [00:10<00:00, 99.44it/s]
Load Images from cataract File
100%|██████████| 1038/1038 [00:09<00:00, 105.23it/s]
Load Images from normal File
100%|██████████| 1074/1074 [00:02<00:00, 499.81it/s]
```

Shuffle The Images

```
Images , Labels = shuffle(Images , Labels , random_state=42)
```

EDA of Images and Get Insights

Number of Images

```
print(f"Number of Total Images {Images.shape[0]}")
print(f"Size of Images {IMAGE_SIZE}")
```

```
Number of Total Images 4217
Size of Images (224, 224)
```

```
Train_Images , dummy_Images , Train_Labels , dummy_Labels = train_test_split(Images , Labels , test_size=0.3, random_state=
Val_Images , Test_Images , Val_Labels , Test_Labels = train_test_split(dummy_Images , dummy_Labels , test_size=0.5, random_
```

```
print(f"Number of Train Images {Train_Images.shape}")
print(f"Number of Train Labels {Train_Labels.shape}")
print(f"Number of Validation Images {Val_Images.shape}")
print(f"Number of Validation Labels {Val_Labels.shape}")
print(f"Number of Test Images {Test_Images.shape}")
print(f"Number of Test Labels {Test_Labels.shape}")
```

```
Number of Train Images (2951, 224, 224, 3)
Number of Train Labels (2951,)
Number of Validation Images (633, 224, 224, 3)
Number of Validation Labels (633,)
Number of Test Images (633, 224, 224, 3)
Number of Test Labels (633,)
```

```
_ , train_counts = np.unique(Train_Labels , return_counts=True)
_ , val_counts = np.unique(Val_Labels , return_counts=True)
_ , test_counts = np.unique(Test_Labels , return_counts=True)

df = pd.DataFrame({
    'Train': train_counts,'Validation': val_counts, 'Test': test_counts
}, index=Class_Names)

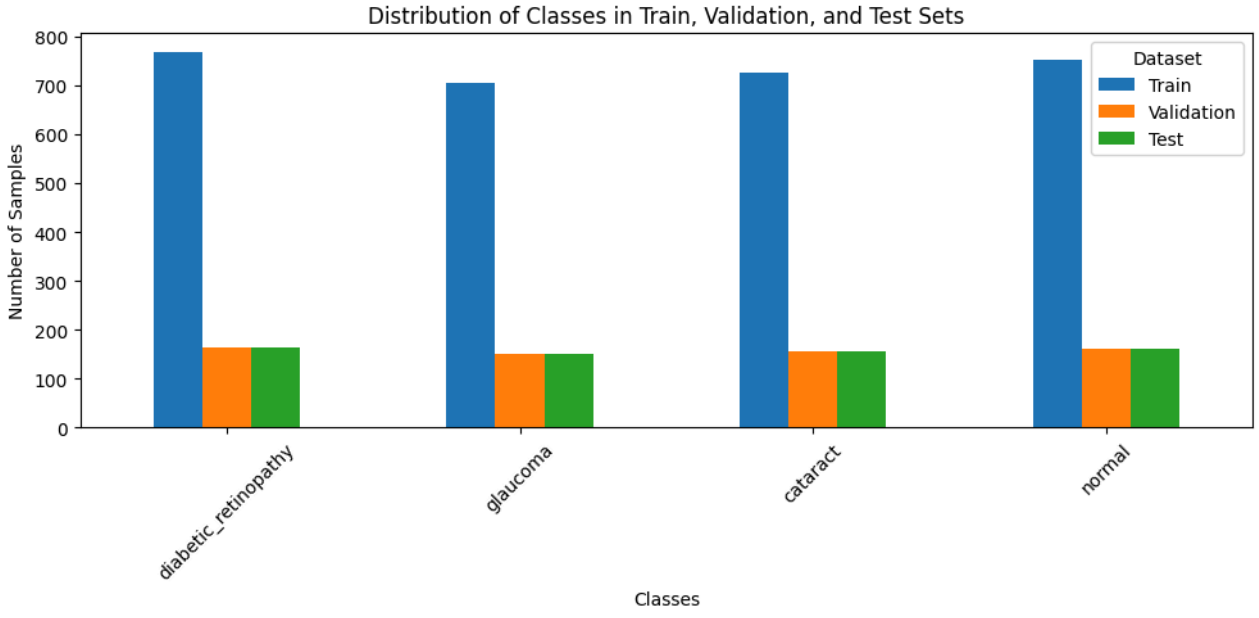
df
```

	Train	Validation	Test
diabetic_retinopathy	768	165	165
glaucoma	705	151	151
cataract	726	156	156
normal	752	161	161

Next steps: Generate code with df | New interactive sheet

```
plt.figure(figsize=(10, 6))
df.plot(kind='bar',figsize=(10,5))
plt.title('Distribution of Classes in Train, Validation, and Test Sets')
plt.xlabel('Classes')
plt.ylabel('Number of Samples')
plt.xticks(rotation=45)
plt.legend(title='Dataset')
plt.tight_layout()
plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```



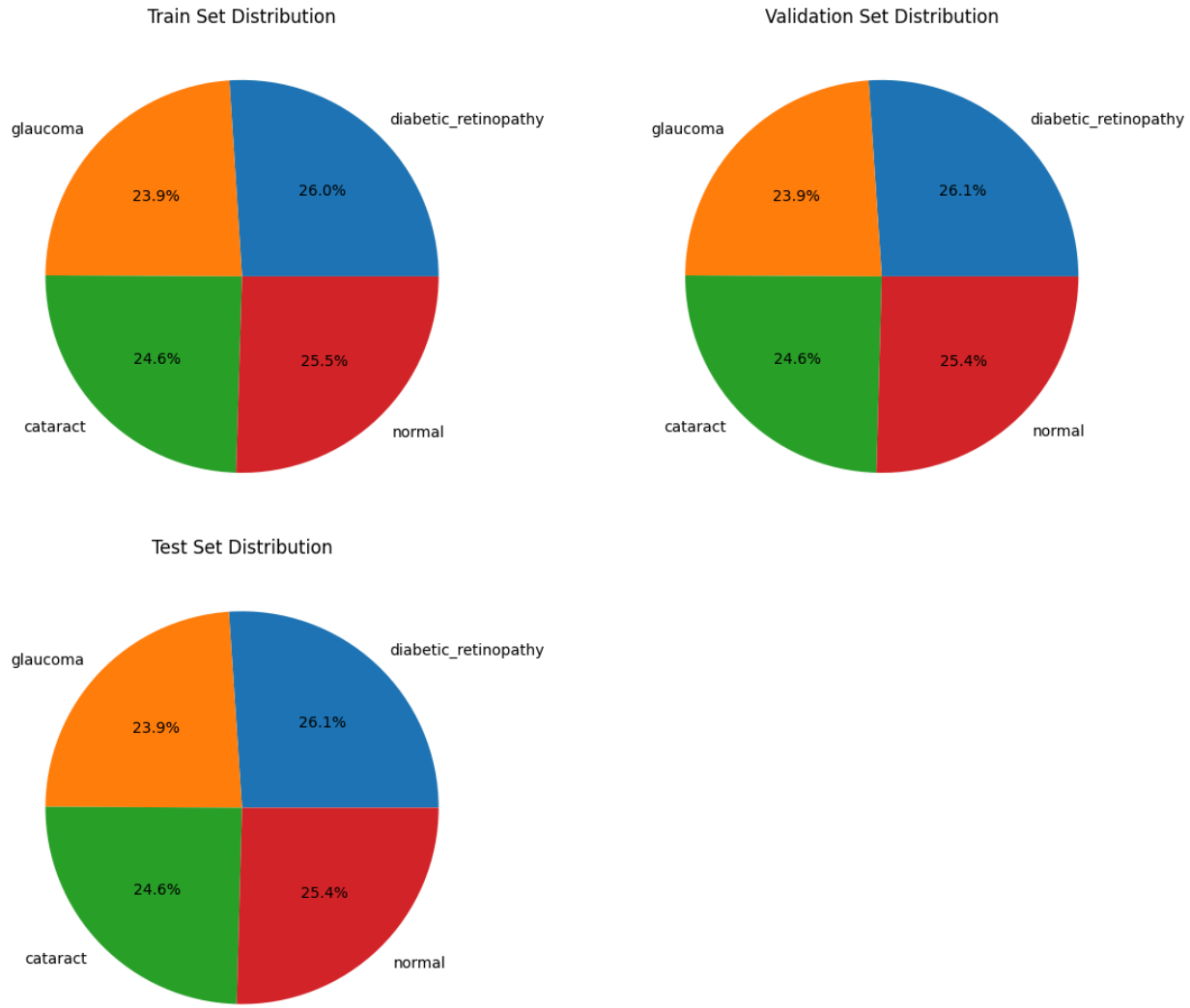

```
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

df.plot.pie(y='Train', autopct='%1.1f%%', title='Train Set Distribution', ax=axs[0,0], legend=False, ylabel='')
df.plot.pie(y='Validation', autopct='%1.1f%%', title='Validation Set Distribution', ax=axs[0,1], legend=False, ylabel='')
```

```
df.plot.pie(y='Test', autopct='%1.1f%%', title='Test Set Distribution', ax=axs[1,0], legend=False, ylabel='')

axs[1,1].axis('off')

plt.tight_layout()
plt.show()
```

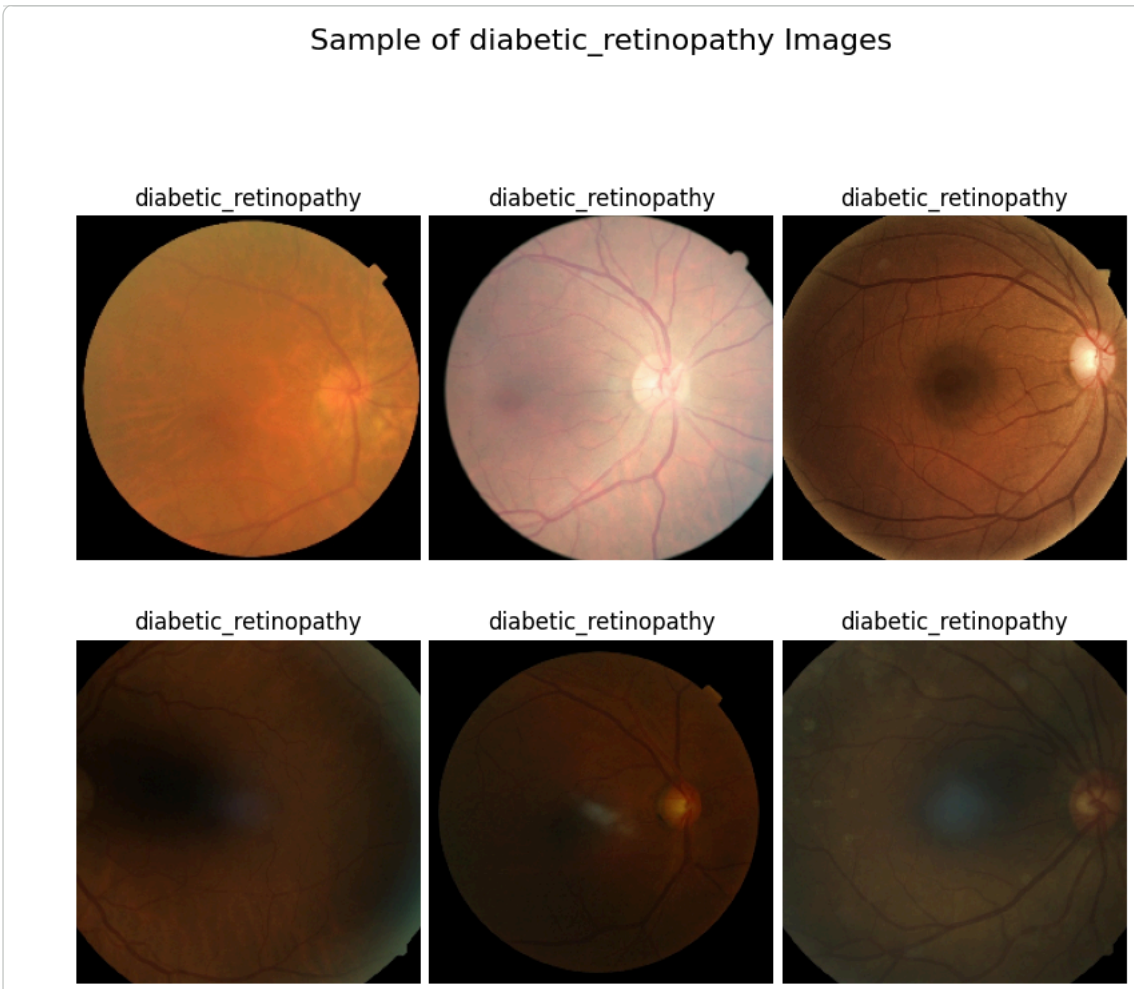


Images In Dataset

```
plt.figure(figsize=(8, 8))
plt.suptitle(f'Sample of {Class_Names[0]} Images', fontsize=16)

for i in range(6):
    plt.subplot(2, 3, i + 1)
    Category_List = np.where(Labels == 0)[0]
    idx = np.random.choice(Category_List)
    plt.imshow(Images[idx] / 255.0)
    plt.axis('off')  # Hide axis for cleaner look
    plt.title(f'{Class_Names[0]}')

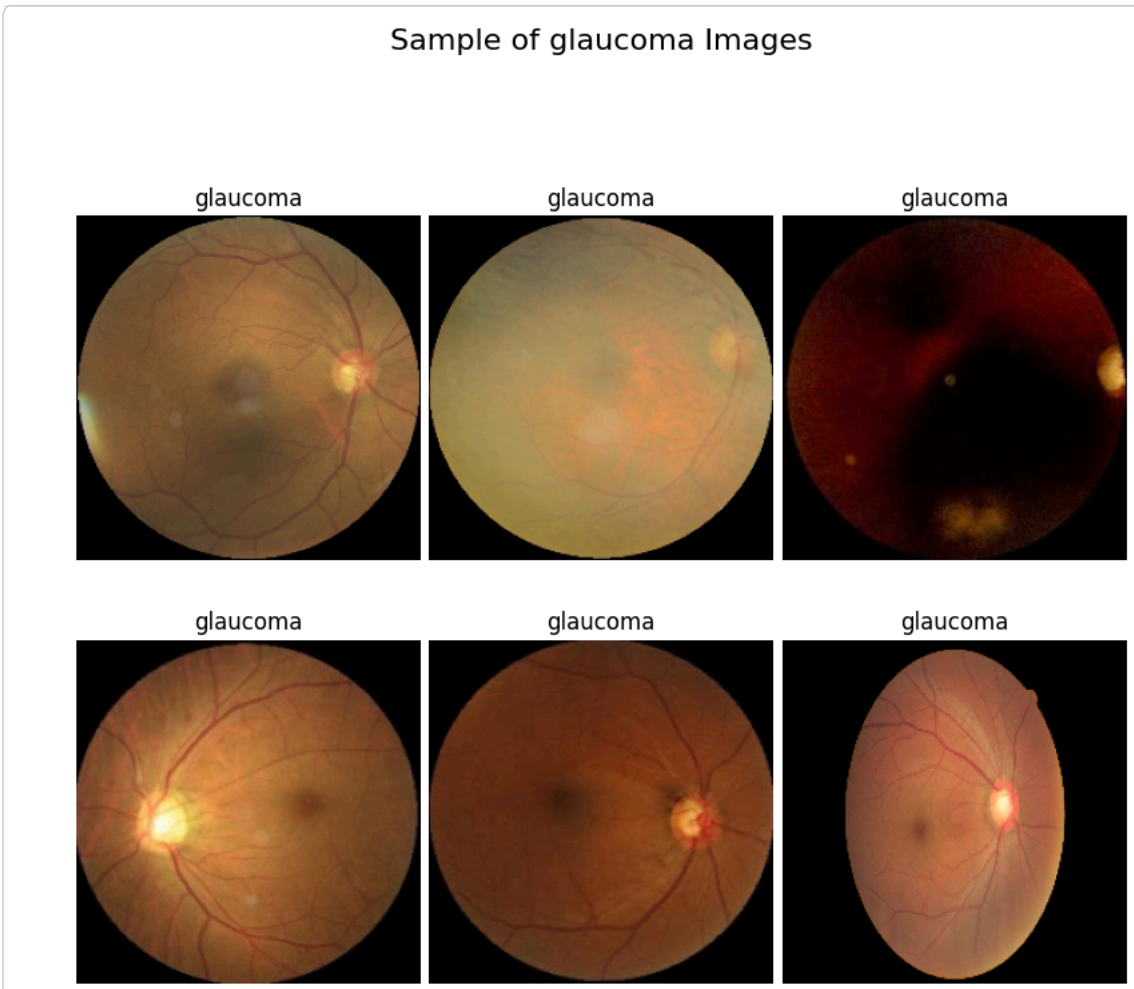
plt.tight_layout(pad=0.5)
plt.subplots_adjust(top=0.88)  # Adjust top to fit the suptitle
plt.show()
```




```
plt.figure(figsize=(8, 8))
plt.suptitle(f'Sample of {Class_Names[1]} Images', fontsize=16)

for i in range(6):
    plt.subplot(2, 3, i + 1)
    Category_List = np.where(Labels == 1)[0]
    idx = np.random.choice(Category_List)
    plt.imshow(Images[idx] / 255.0)
    plt.axis('off')
    plt.title(f'{Class_Names[1]}')

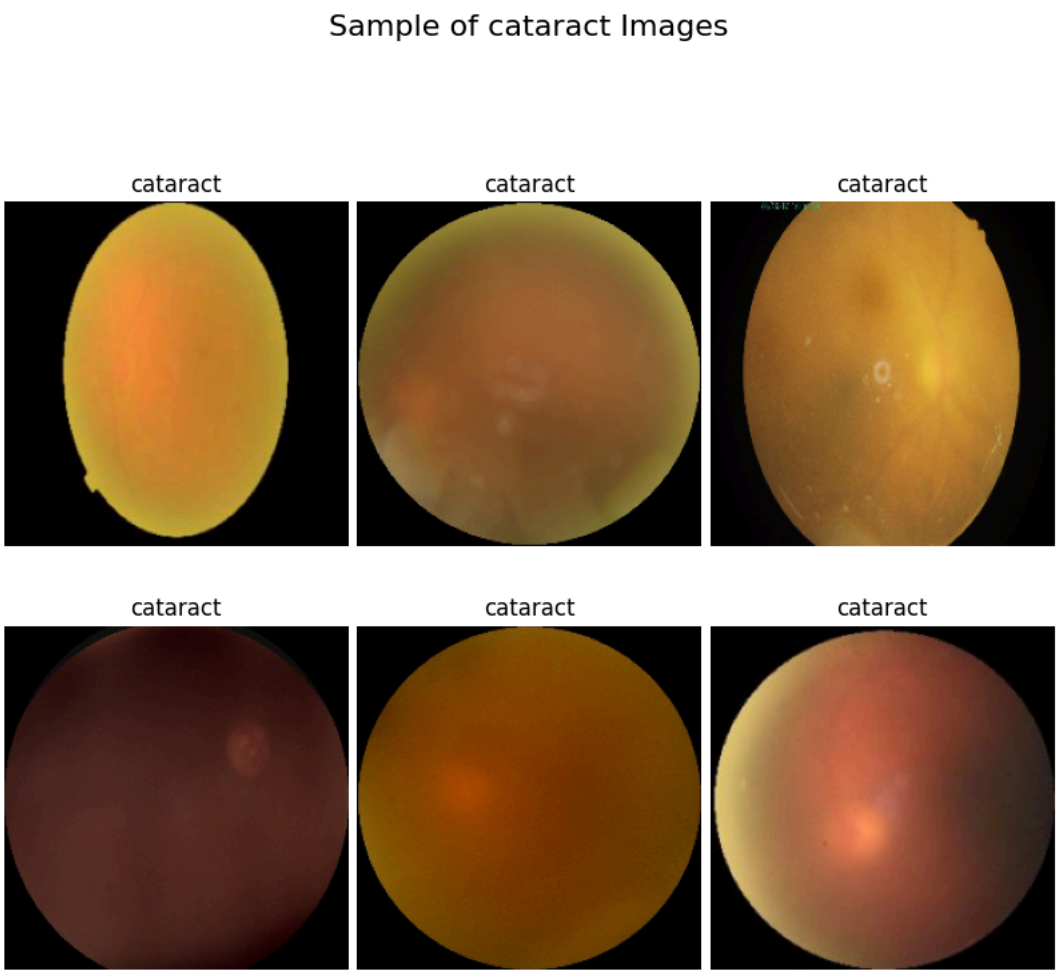
plt.tight_layout(pad=0.5)
plt.subplots_adjust(top=0.88)
plt.show()
```




```
plt.figure(figsize=(8, 8))
plt.suptitle(f'Sample of {Class_Names[2]} Images', fontsize=16)

for i in range(6):
    plt.subplot(2, 3, i + 1)
    Category_List = np.where(Labels == 2)[0]
    idx = np.random.choice(Category_List)
    plt.imshow(Images[idx] / 255.0)
    plt.axis('off')
    plt.title(f'{Class_Names[2]}')

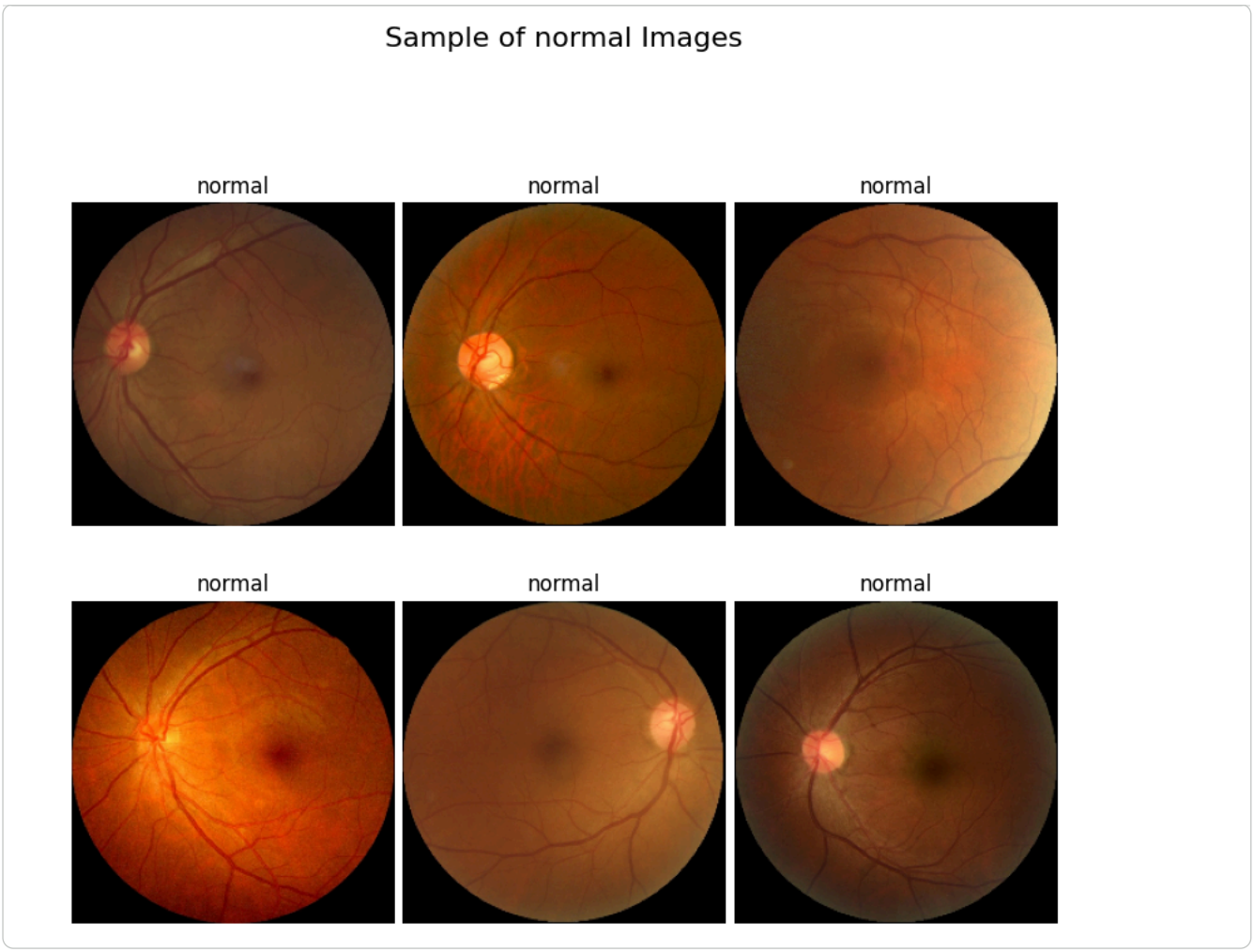
plt.tight_layout(pad=0.5)
plt.subplots_adjust(top=0.88)
plt.show()
```




```
plt.figure(figsize=(8, 8))
plt.suptitle(f'Sample of {Class_Names[3]} Images', fontsize=16)

for i in range(6):
    plt.subplot(2, 3, i + 1)
    Category_List = np.where(Labels == 3)[0]
    idx = np.random.choice(Category_List)
    plt.imshow(Images[idx] / 255.0)
    plt.axis('off')
    plt.title(f'{Class_Names[3]}')

plt.tight_layout(pad=0.5)
plt.subplots_adjust(top=0.88)
plt.show()
```



Model Custom CNN

Build Custom CNN

```
# ============================================================
# DATA AUGMENTATION AND DATA GENERATORS
# ============================================================

# Training data augmentation
# Keeps the original augmentation strategy to preserve model performance
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255,
    rotation_range=40,
    horizontal_flip=True
)

# Validation data should NOT be augmented
val_datagen = ImageDataGenerator(
    rescale=1.0 / 255
)

# Test data should NOT be augmented
test_datagen = ImageDataGenerator(
    rescale=1.0 / 255
)


# ============================================================
# TRAIN GENERATOR
# ============================================================

train_generator = train_datagen.flow(
    x=Train_Images,
    y=Train_Labels,
    batch_size=BATCH_SIZE,
    shuffle=True
)
```

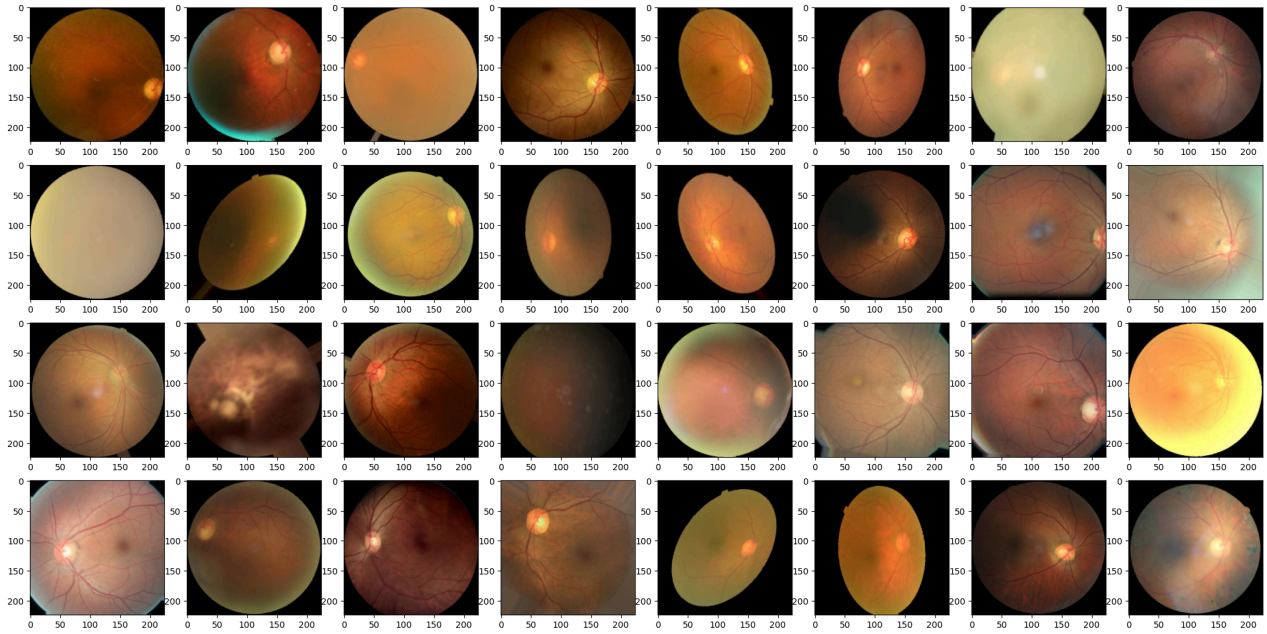
```
# ============================================================
# VALIDATION GENERATOR
# ============================================================

val_generator = val_datagen.flow(
    x=Val_Images,
    y=Val_Labels,
    batch_size=BATCH_SIZE,
    shuffle=True
)


# ============================================================
# TEST GENERATOR
# ============================================================

test_generator = test_datagen.flow(
    x=Test_Images,
    y=Test_Labels,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

```
for x , y in train_generator:
    plt.figure(figsize=(20,20))
    for i in range(32):
        plt.subplot(8,8,i+1)
        plt.imshow(x[i])
        plt.tight_layout(pad=0 , h_pad=0 , w_pad=0)
    break
```



```
# ============================================================================
# BUILD CUSTOM CNN
# ============================================================================

KerasModel = keras.models.Sequential([

    # ------------------------------------------------------------------------
    # INPUT
    #
```

```
keras.layers.Input(
    shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], CHANNELS)
),


# ========================================================================
# BLOCK 1
# ========================================================================

keras.layers.Conv2D(
    32,
    (3, 3),
    activation='relu',
    padding='same',
    name='conv1_1'
),

keras.layers.BatchNormalization(
    name='bn1_1'
),

keras.layers.Conv2D(
    32,
    (3, 3),
    activation='relu',
    padding='same',
    name='conv1_2'
),

keras.layers.BatchNormalization(
    name='bn1_2'
),

keras.layers.MaxPooling2D(
    (2, 2),
    name='pool1'
),

keras.layers.Dropout(
    0.10,
    name='dropout1'
),


# ========================================================================
# BLOCK 2
# ========================================================================

keras.layers.Conv2D(
    64,
    (3, 3),
    activation='relu',
    padding='same',
    name='conv2_1'
),

keras.layers.BatchNormalization(
    name='bn2_1'
),

keras.layers.Conv2D(
    64,
    (3, 3),
    activation='relu',
    padding='same',
    name='conv2_2'
),

keras.layers.BatchNormalization(
    name='bn2_2'
),

keras.layers.MaxPooling2D(
    (2, 2),
    name='pool2'
),

keras.layers.Dropout(
    0.15,
    name='dropout2'
),
```

```
# ============================================================================
# BLOCK 3
# ============================================================================

keras.layers.Conv2D(
    128,
    (3, 3),
    activation='relu',
    padding='same',
    name='conv3_1'
),

keras.layers.BatchNormalization(
    name='bn3_1'
),

keras.layers.Conv2D(
    128,
    (3, 3),
    activation='relu',
    padding='same',
    name='conv3_2'
),

keras.layers.BatchNormalization(
    name='bn3_2'
),

keras.layers.MaxPooling2D(
    (2, 2),
    name='pool3'
),

keras.layers.Dropout(
    0.20,
    name='dropout3'
),


# ============================================================================
# FINAL CONVOLUTIONAL BLOCK
# ============================================================================
# Keep this layer because your Streamlit Grad-CAM implementation can
# explicitly target it.
#
# Feature map:
# 28 x 28 x 256
# ============================================================================

keras.layers.Conv2D(
    256,
    (3, 3),
    activation='relu',
    padding='same',
    name='gradcam_conv'
),

keras.layers.BatchNormalization(
    name='gradcam_bn'
),


# ============================================================================
# CLASSIFICATION HEAD
# ============================================================================
#
# IMPORTANT:
#
# We intentionally use Flatten here instead of GlobalAveragePooling2D.
#
# Feature map:
# 28 x 28 x 256 = 200,704 features
#
# Dense(160):
#
# 200,704 x 160 = 32,112,640 weights
#
# This gives the model substantially more capacity than your previous
# 7 MB model while avoiding the enormous 100M+ parameter Dense(512)
# layer from the original model.
# ============================================================================
```

```
    keras.layers.Flatten(
        name='flatten'
    ),

    keras.layers.Dense(
        256,
        activation='relu',
        name='dense_features'
    ),

    keras.layers.BatchNormalization(
        name='dense_bn'
    ),

    keras.layers.Dropout(
        0.35,
        name='classification_dropout'
    ),

    keras.layers.Dense(
        4,
        activation='softmax',
        dtype='float32',
        name='predictions'
    )
])


# ==============================================================================
# DISPLAY MODEL ARCHITECTURE
# ==============================================================================

KerasModel.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv1_1 (Conv2D)	(None, 224, 224, 32)	896
bn1_1 (BatchNormalization)	(None, 224, 224, 32)	128
conv1_2 (Conv2D)	(None, 224, 224, 32)	9,248
bn1_2 (BatchNormalization)	(None, 224, 224, 32)	128
pool1 (MaxPooling2D)	(None, 112, 112, 32)	0
dropout1 (Dropout)	(None, 112, 112, 32)	0
conv2_1 (Conv2D)	(None, 112, 112, 64)	18,496
bn2_1 (BatchNormalization)	(None, 112, 112, 64)	256
conv2_2 (Conv2D)	(None, 112, 112, 64)	36,928
bn2_2 (BatchNormalization)	(None, 112, 112, 64)	256
pool2 (MaxPooling2D)	(None, 56, 56, 64)	0
dropout2 (Dropout)	(None, 56, 56, 64)	0
conv3_1 (Conv2D)	(None, 56, 56, 128)	73,856
bn3_1 (BatchNormalization)	(None, 56, 56, 128)	512
conv3_2 (Conv2D)	(None, 56, 56, 128)	147,584
bn3_2 (BatchNormalization)	(None, 56, 56, 128)	512
pool3 (MaxPooling2D)	(None, 28, 28, 128)	0
dropout3 (Dropout)	(None, 28, 28, 128)	0
gradcam_conv (Conv2D)	(None, 28, 28, 256)	295,168
gradcam_bn (BatchNormalization)	(None, 28, 28, 256)	1,024
flatten (Flatten)	(None, 200704)	0
dense_features (Dense)	(None, 256)	51,380,480
dense_bn (BatchNormalization)	(None, 256)	1,024
classification_dropout (Dropout)	(None, 256)	0
predictions (Dense)	(None, 4)	1,028

```
# ============================================================================
# COMPILE CUSTOM CNN
# ============================================================================

KerasModel.compile(
    optimizer=Adam(
        learning_rate=0.0005
    ),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# ============================================================================
# TRAINING CALLBACKS
# ============================================================================

# ----------------------------------------------------------------------------
# 1. SAVE THE BEST MODEL
# ----------------------------------------------------------------------------
# We monitor validation accuracy because the main goal is classification
# performance rather than simply minimizing the loss.
# ----------------------------------------------------------------------------

best_model_callback = keras.callbacks.ModelCheckpoint(
    'best_model.h5',
    monitor='val_accuracy',
    mode='max',
    save_best_only=True,
    verbose=1
)
```

```
# ------------------------------------------------------------------------
# 2. EARLY STOPPING
# ------------------------------------------------------------------------
# If validation accuracy does not improve for 15 epochs, training stops.
# The weights from the best validation-accuracy epoch are restored.
# ------------------------------------------------------------------------

early_stopping_callback = keras.callbacks.EarlyStopping(
    monitor='val_accuracy',
    mode='max',
    patience=15,
    restore_best_weights=True,
    verbose=1
)


# ------------------------------------------------------------------------
# 3. REDUCE LEARNING RATE WHEN VALIDATION ACCURACY STOPS IMPROVING
# ------------------------------------------------------------------------

learning_rate_reduction = keras.callbacks.ReduceLROnPlateau(
    monitor='val_accuracy',
    mode='max',
    factor=0.5,
    patience=4,
    verbose=1,
    min_lr=1e-6
)


# ========================================================================
# TRAIN THE MODEL
# ========================================================================

history = KerasModel.fit(
    train_generator,

    epochs=200,

    validation_data=val_generator,

    callbacks=[
        best_model_callback,
        early_stopping_callback,
        learning_rate_reduction
    ],

    verbose=1
)
```

```
93/93 ━━━━━━━━━━━━━━━━━━━━ 40s 428ms/step - accuracy: 0.8868 - loss: 0.2967 - val_accuracy: 0.8389 - val_loss: 0.4271 - le
Epoch 57/200
93/93 ━━━━━━━━━━━━━━━━━━━━ 0s 416ms/step - accuracy: 0.8895 - loss: 0.2918
Epoch 57: val_accuracy did not improve from 0.84518
93/93 ━━━━━━━━━━━━━━━━━━━━ 40s 426ms/step - accuracy: 0.8817 - loss: 0.2995 - val_accuracy: 0.8389 - val_loss: 0.4466 - le
Epoch 58/200
93/93 ━━━━━━━━━━━━━━━━━━━━ 0s 413ms/step - accuracy: 0.8795 - loss: 0.3101
Epoch 58: val_accuracy did not improve from 0.84518

Epoch 58: ReduceLROnPlateau reducing learning rate to 3.906250185536919e-06.
93/93 ━━━━━━━━━━━━━━━━━━━━ 39s 423ms/step - accuracy: 0.8861 - loss: 0.2875 - val_accuracy: 0.8136 - val_loss: 0.4884 - le
Epoch 59/200
93/93 ━━━━━━━━━━━━━━━━━━━━ 0s 409ms/step - accuracy: 0.8811 - loss: 0.2920
Epoch 59: val_accuracy did not improve from 0.84518
93/93 ━━━━━━━━━━━━━━━━━━━━ 39s 420ms/step - accuracy: 0.8756 - loss: 0.2958 - val_accuracy: 0.8167 - val_loss: 0.4846 - le
Epoch 60/200
93/93 ━━━━━━━━━━━━━━━━━━━━ 0s 417ms/step - accuracy: 0.8776 - loss: 0.3014
Epoch 60: val_accuracy did not improve from 0.84518
93/93 ━━━━━━━━━━━━━━━━━━━━ 40s 431ms/step - accuracy: 0.8753 - loss: 0.3067 - val_accuracy: 0.8152 - val_loss: 0.4765 - le
Epoch 61/200
93/93 ━━━━━━━━━━━━━━━━━━━━ 0s 426ms/step - accuracy: 0.8754 - loss: 0.2837
Epoch 61: val_accuracy did not improve from 0.84518
93/93 ━━━━━━━━━━━━━━━━━━━━ 41s 436ms/step - accuracy: 0.8831 - loss: 0.2899 - val_accuracy: 0.8167 - val_loss: 0.4722 - le
Epoch 61: early stopping
Restoring model weights from the end of the best epoch: 46.
```

Summary of KerasModel

```
print(KerasModel.summary())
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv1_1 (Conv2D)	(None, 224, 224, 32)	896
bn1_1 (BatchNormalization)	(None, 224, 224, 32)	128
conv1_2 (Conv2D)	(None, 224, 224, 32)	9,248
bn1_2 (BatchNormalization)	(None, 224, 224, 32)	128
pool1 (MaxPooling2D)	(None, 112, 112, 32)	0
dropout1 (Dropout)	(None, 112, 112, 32)	0
conv2_1 (Conv2D)	(None, 112, 112, 64)	18,496
bn2_1 (BatchNormalization)	(None, 112, 112, 64)	256
conv2_2 (Conv2D)	(None, 112, 112, 64)	36,928
bn2_2 (BatchNormalization)	(None, 112, 112, 64)	256
pool2 (MaxPooling2D)	(None, 56, 56, 64)	0
dropout2 (Dropout)	(None, 56, 56, 64)	0
conv3_1 (Conv2D)	(None, 56, 56, 128)	73,856
bn3_1 (BatchNormalization)	(None, 56, 56, 128)	512
conv3_2 (Conv2D)	(None, 56, 56, 128)	147,584
bn3_2 (BatchNormalization)	(None, 56, 56, 128)	512
pool3 (MaxPooling2D)	(None, 28, 28, 128)	0
dropout3 (Dropout)	(None, 28, 28, 128)	0
gradcam_conv (Conv2D)	(None, 28, 28, 256)	295,168
gradcam_bn (BatchNormalization)	(None, 28, 28, 256)	1,024
flatten (Flatten)	(None, 200704)	0
dense_features (Dense)	(None, 256)	51,380,480
dense_bn (BatchNormalization)	(None, 256)	1,024
classification_dropout (Dropout)	(None, 256)	0
predictions (Dense)	(None, 4)	1,028

Total params: 155,898,734 (594.71 MB)
Trainable params: 51,965,604 (198.23 MB)

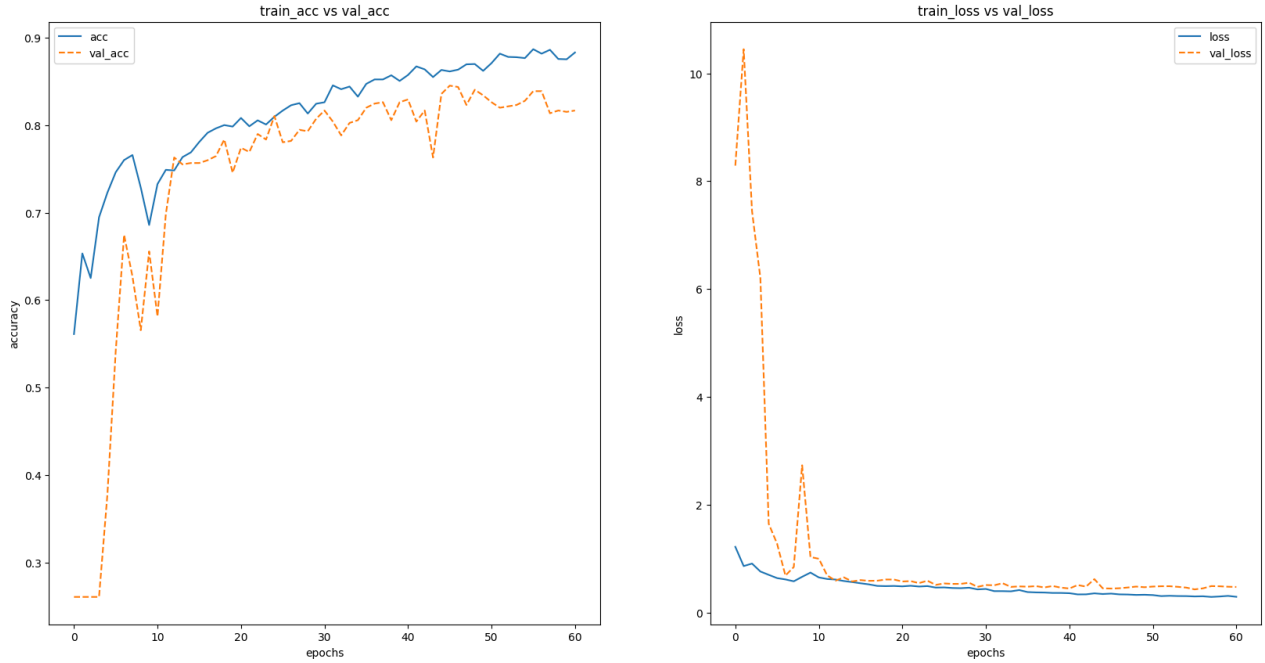
Evaluate Custom CNN

```
def plot_accuracy_loss(history):
    """
        Plot the accuracy and the loss during the training of the nn.
    """
    fig = plt.figure(figsize=(20,10))

    # Plot accuracy
    plt.subplot(121)
    plt.plot(history.history['accuracy'],'-', label = "acc")
    plt.plot(history.history['val_accuracy'], '--', label = "val_acc")
    plt.title("train_acc vs val_acc")
    plt.ylabel("accuracy")
    plt.xlabel("epochs")
    plt.legend()

    # Plot loss function
    plt.subplot(122)
    plt.plot(history.history['loss'],'-', label = "loss")
    plt.plot(history.history['val_loss'], '--', label = "val_loss")
    plt.title("train_loss vs val_loss")
    plt.ylabel("loss")
    plt.xlabel("epochs")

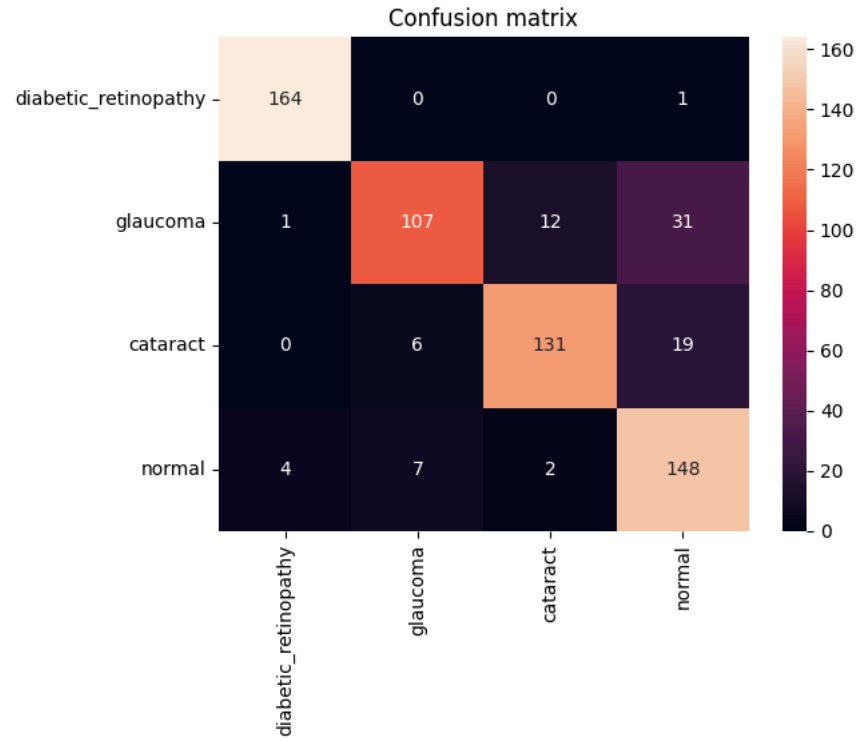
    plt.legend()
    plt.show()
plot_accuracy_loss(history)
```




```
model = keras.models.load_model('best_model.h5')
predictions = model.predict(test_generator)     # Vector of probabilities
pred_labels = np.argmax(predictions, axis = 1) # We take the highest probability
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty
20/20 ━━━━━━━━━━━━━━━━━━━━ 7s 211ms/step
```

```
CM = confusion_matrix(Test_Labels, pred_labels)
ax = plt.axes()
sns.heatmap(CM, annot=True,
           annot_kws={"size": 10},
           xticklabels = Class_Names,
           yticklabels = Class_Names,
           ax = ax,
           fmt='d')
ax.set_title('Confusion matrix')
plt.show()
```




```
print(classification_report(Test_Labels, pred_labels,target_names = Class_Names))
```

```
                     precision    recall  f1-score   support

diabetic_retinopathy       0.97      0.99      0.98       165
            glaucoma       0.89      0.71      0.79       151
            cataract       0.90      0.84      0.87       156
              normal       0.74      0.92      0.82       161

            accuracy                           0.87       633
           macro avg       0.88      0.87      0.87       633
        weighted avg       0.88      0.87      0.87       633
```

ROC Curves for Custom CNN Model

```
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for ROC curve calculation
n_classes = len(Class_Names)
Test_Labels_binarized = label_binarize(Test_Labels, classes=range(n_classes))

# Compute ROC curve and ROC area for each class
fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(Test_Labels_binarized[:, i], predictions[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

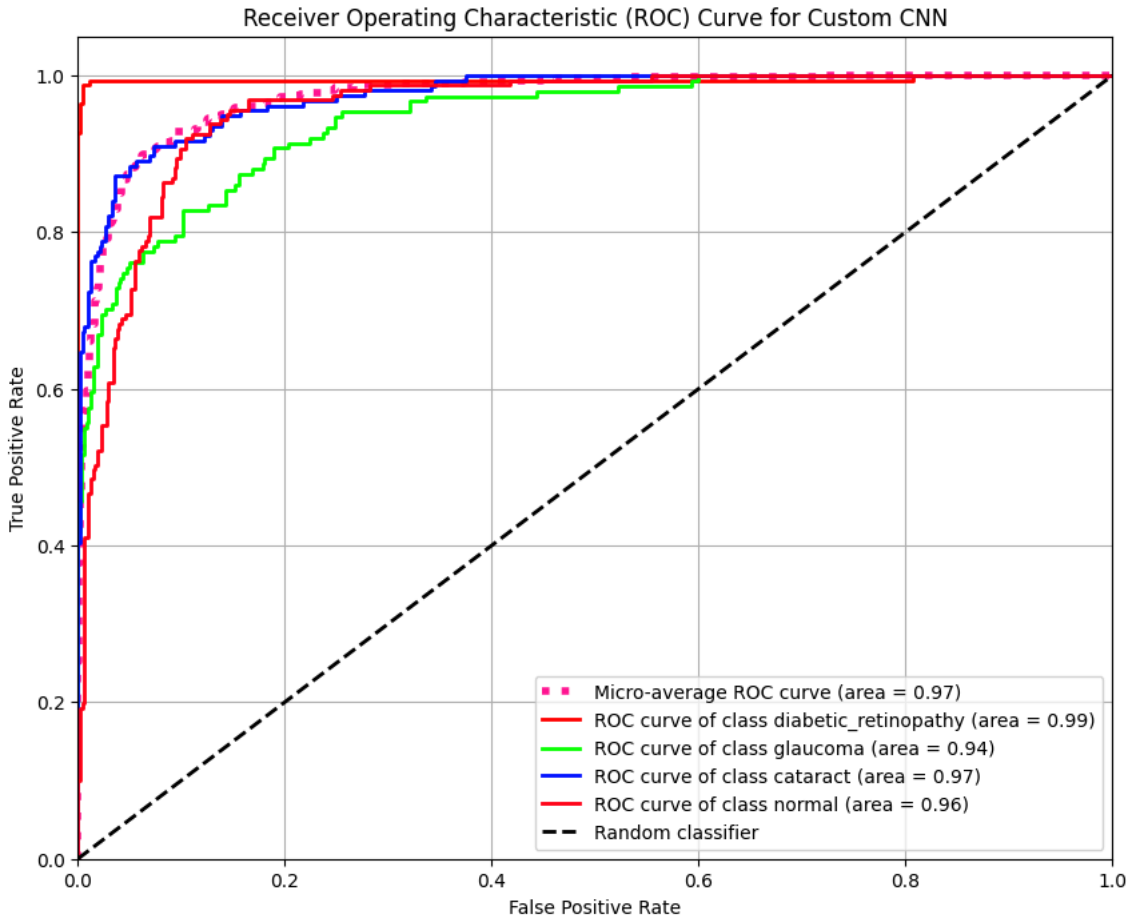
# Compute micro-average ROC curve and ROC area
fpr["micro"], tpr["micro"], _ = roc_curve(Test_Labels_binarized.ravel(), predictions.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

# Plot ROC curves
plt.figure(figsize=(10, 8))
plt.plot(fpr["micro"], tpr["micro"],
         label=f'Micro-average ROC curve (area = {roc_auc["micro"]:0.2f})',
```

```
          color='deeppink', linestyle=':', linewidth=4)

colors = plt.cm.get_cmap('hsv', n_classes)
for i, color in zip(range(n_classes), colors(np.linspace(0, 1, n_classes))):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label=f'ROC curve of class {Class_Names[i]} (area = {roc_auc[i]:0.2f})')

plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Random classifier')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve for Custom CNN')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



Start coding or generate with AI.

📊 Classification Report – Observations and Insights

The trained CNN model achieved an overall test accuracy of 86.89%, demonstrating good performance in classifying retinal images into four categories: diabetic retinopathy, glaucoma, cataract, and normal. The model obtained a macro-average precision of 0.88, recall of 0.87, and F1-score of 0.87, indicating relatively balanced overall performance across the four classes.

🔬 Key Observations:

- Diabetic Retinopathy achieved the best performance, with a precision of 0.97, recall of 0.99, and F1-score of 0.98. This indicates that the model was highly effective at identifying diabetic retinopathy images and produced very few missed cases in the test set.
- Glaucoma was the most challenging class for the model. Although it achieved a relatively high precision of 0.89, its recall was 0.71, resulting in an F1-score of 0.79. This suggests that while most images predicted as glaucoma were actually glaucoma, a portion of the actual glaucoma cases were classified as other categories.
- Cataract showed good and relatively balanced performance, achieving 0.90 precision, 0.84 recall, and 0.87 F1-score. The model was generally effective at distinguishing cataract images from the other classes.
- Normal images achieved a high recall of 0.92, indicating that the model successfully identified most of the truly normal retinal images. However, its precision was comparatively lower at 0.74, suggesting that some images predicted as normal actually belonged to disease categories.

📈 Overall Insight

The relatively small difference between the macro-average F1-score (0.87) and weighted-average F1-score (0.87) indicates that the model's performance is reasonably consistent across the dataset and is not being dominated by a single class. The test dataset also contains a fairly balanced number of samples across the four categories, with support ranging from 151 to 165 images per class.

Overall, the results demonstrate that the CNN has good classification capability, particularly for diabetic retinopathy and cataract. The comparatively lower recall for glaucoma represents the primary area where classification errors occur. However, considering the overall accuracy of 86.89% and macro F1-score of 0.87, the model provides a strong result for an academic deep-learning project.

```
from sklearn.metrics import precision_recall_fscore_support
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate per-class metrics
precision, recall, f1_score, _ = precision_recall_fscore_support(
    Test_Labels, pred_labels, labels=np.arange(len(Class_Names))
)

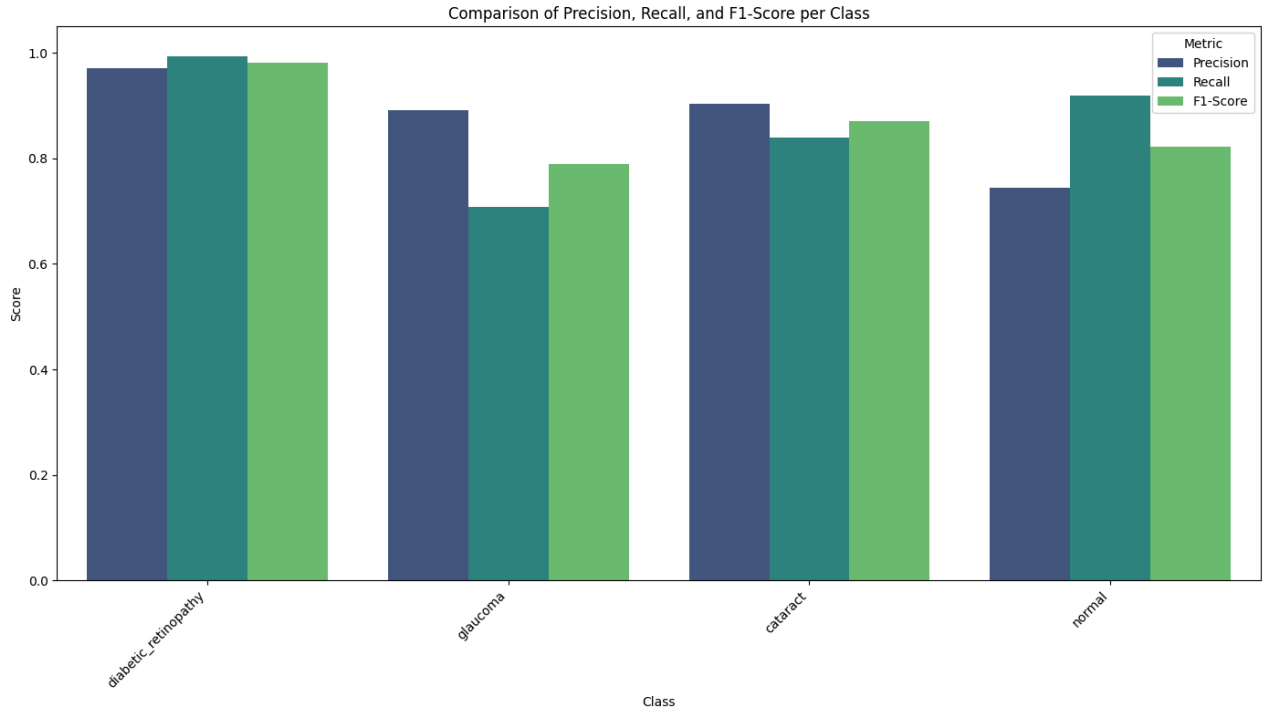
# Get overall accuracy
accuracy = (pred_labels == Test_Labels).mean()

# Create a DataFrame for easier plotting
metrics_df = pd.DataFrame({
    'Class': Class_Names,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1_score
})

# Melt the DataFrame for seaborn barplot
metrics_melted = metrics_df.melt(id_vars='Class', var_name='Metric', value_name='Score')

plt.figure(figsize=(14, 8))
sns.barplot(x='Class', y='Score', hue='Metric', data=metrics_melted, palette='viridis')
plt.title('Comparison of Precision, Recall, and F1-Score per Class')
plt.ylabel('Score')
plt.xlabel('Class')
plt.ylim(0, 1.05)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

print(f"Overall Accuracy: {accuracy:.2f}")
```




```
Overall Accuracy: 0.87
```

Predicting with Custom CNN

```
plt.figure(figsize=(15,20))
plt.suptitle(f"Sample Of Images Predicted by KerasModel" , fontsize=16)

for i in range(24):
    plt.subplot(6,5,i+1)
    idx = np.random.choice(np.arange(len(pred_labels)))
    plt.imshow(Test_Images[idx] / 255.0)
    plt.xlabel(f'Pred {Class_Names[pred_labels[idx]]}')
    plt.ylabel(f'True {Class_Names[Test_Labels[idx]]}')
    plt.tight_layout(pad=0 , h_pad=0 , w_pad=2)
```

Download Custom CNN Model

```
# To download the custom CNN model file (.h5) to your local machine:
from google.colab import files

files.download('best_model.h5')
```

Pretrained Modeling

Optimizations for EfficientNet

To address long training times and prevent runtime disconnections, especially with larger models like EfficientNet, I'm making the following changes:

1. **Mixed Precision Training**: This will use `float16` for most computations to reduce memory usage and speed up training.
2. **Smaller EfficientNet Variant**: Switching from `EfficientNetB7` to `EfficientNetB0` drastically reduces model size and computational demands while often providing strong performance.
3. **Redefine Constants and Imports**: Re-defining `IMAGE_SIZE`, `BATCH_SIZE`, `CHANNELS`, and re-importing `ImageDataGenerator` and `callbacks` ensures these are available even if the runtime restarts partially.
4. **Learning Rate Scheduler**: Adding `ReduceLROnPlateau` for EfficientNet to dynamically adjust the learning rate during training, which can help in faster convergence and prevent plateaus.

Note: Your Custom CNN model already includes `ModelCheckpoint` and `EarlyStopping` for saving the best model based on validation loss.

```
import tensorflow as tf

# Enable mixed precision training for faster training and reduced memory usage
tf.keras.mixed_precision.set_global_policy('mixed_float16')
```